GenAI SQL Refactoring Assistant – Milestone 4

RAG + Experimental Optimisation

Focus

Context-aware SQL refactoring

Method

Retrieval-Augmented Generation + Iterative Experiments

Milestone Objective

Goal of Milestone 4

1. Improve SQL refactoring quality beyond baseline LLM
2. Introduce **context-awareness using RAG**
3. Perform **systematic experimentation** to optimise performance

Why RAG?

Problem with Baseline Model

- Lacks schema awareness
- No access to SQL best practices
- Produces generic refactoring

Solution

Use **Retrieval-Augmented Generation**

- Inject external knowledge dynamically

RAG Step 1 – Knowledge Base Creation

Sources Used

- SQLite documentation
- HiveQL documentation
- Database schema

Processing

- Text cleaning and chunking
- Converted into smaller semantic units

Observation

Some documents missing → fallback content used

Inference

Quality of KB directly impacts final outputs

RAG Step 2 – Embeddings + Vector Store

Embedding Model

`all-MiniLM-L6-v2`

Vector Store

FAISS for similarity search

What happens

- Each chunk → vector representation
- Stored for fast retrieval

Observation

Efficient indexing enables scalable retrieval

RAG Step 3 – Retriever + Reranker

Retriever

Fetches top-k similar chunks

Reranker

Cross-encoder

- `ms-marco-MiniLM-L-6-v2`

Reorders results based on relevance

Why Reranking?

Improves context quality

Inference: Better context → better generation

RAG Dataset Construction

Pipeline

01

Input SQL query

02

Retrieve top-k chunks

03

Rerank

04

Build context string

05

Append to input

Result

Two datasets created:

- Without RAG
- With RAG

Observation

RAG dataset creation is computationally expensive

Experiment Strategy

Approach

→ Start with baseline

→ Modify one factor at a time

→ Track performance

Evaluation

Validation exact match

Output quality

Experiment 1 – Baseline Model

Setup

- T5-base + LoRA
- No RAG

Observation

~223M

Parameters

0.39%

Trainable

Inference: Efficient training but limited contextual understanding

Experiment 2 – Learning Rate Tuning

Changes

Tested different learning rates

High LR	Low LR
Unstable training	Slower convergence

Inference: Optimal LR critical for stable learning

Experiment 3 – Scheduler Variants

Schedulers Tested

- Linear
- Cosine
- Constant

Observation

Cosine decay showed smoother convergence

Inference

Scheduler impacts training dynamics significantly

Experiment 4 – Beam Search Width

2	4	6	8
Beam Size	Beam Size	Beam Size	Beam Size

Observation

Larger beams → better outputs but slower

Inference

Trade-off: quality vs latency

Experiment 5 – Optimizer Comparison

AdamW	Adafactor
More stable	Memory efficient

Inference: Choice depends on resource constraints

Experiment 6 – Regularisation

Techniques

- Weight decay
- Dropout
- Label smoothing

Observation

- Reduced overfitting
- Improved generalisation

Inference

Regularisation crucial for robustness

Experiment 7 – RAG Integration

Major Change

Added retrieved context to inputs

Setup

Top-3 retrieved + reranked chunks

Observation

Outputs more structured and relevant

Inference

RAG improves semantic understanding

Experiment 8 – RAG Ablation Study

Retriever Only

Retriever + Reranker

Observation

Reranker improves relevance

Inference

Retrieval quality matters more than quantity

Experiment 9 – Final Optimised Model

Combined

- Best LR
- Best Scheduler
- Best Regularisation
- RAG + Reranker

Goal: Maximise validation performance

Final Model Performance

1	2	3
Evaluation Metric	**Output**	**Saved Results**
Validation exact match	Best model selected	`milestone4_experiment_log.csv`

Key Observations

1. Baseline model lacks context
2. Hyperparameter tuning improves stability
3. RAG significantly improves output relevance
4. Reranker enhances retrieval quality

Final Conclusion

Best Performing Setup

- Tuned T5 + LoRA
- RAG (top-k retrieval)
- Cross-encoder reranker

Final Insight 👉

Performance = **Model** + **Data** + **Retrieval Quality**